

BUSFIN 8200 Problem Sets: AI Policy

What you need to deliver for each problem set:

1. **The problem set solution in PDF format** (not handwritten).
2. **A Git repository containing all files and results.** This must include the source files used to generate the PDF (e.g., the `.tex` file) as well as all coding scripts used to produce the results (e.g., `.R` or `.py` files). I need to be able to reproduce both your results and your PDF from the source files in your Git repository.

AI Policy

AI is revolutionizing research production. At the same time, the researchers who benefit most from AI are those who understand the economics, mathematics, and empirical methods underlying what they ask AI to do. My goal is therefore not to prevent you from using AI, but to constrain how you use it on problem sets so that AI accelerates your learning rather than replaces it. Earlier cohorts learned these tools by working through similar problems without AI. You should leave this course with at least the same underlying understanding, while also learning how to use AI productively.

If you use AI tools while solving a problem set, follow the process below.

1. How to use AI

For each task (i.e., each item within a question), first determine conceptually what the task requires. You should then follow the appropriate procedure below.

(a) Mathematical questions

First work through the derivation yourself, either by hand or directly in your `.tex` file (or equivalent). If you work by hand, you may upload a picture of your derivation to AI and ask it to translate what you wrote into LaTeX.

Once you have produced your own derivation, you may ask AI to check it. If AI identifies a possible error, you may ask it to explain **why a particular step is incorrect**, but you must determine and implement the correction yourself. Do not ask AI to rewrite, complete, or replace your derivation.

If you become stuck before completing the derivation, you may show AI the work you have already done and ask it to diagnose a specific problem with your reasoning or a particular step. You may not ask it to continue the derivation or provide the remaining solution.

AI should therefore serve as a **checker and diagnostic tool**, not as the source of the mathematical derivation.

After you have written the substantive derivation yourself, you may ask AI to improve exposition, grammar, or LaTeX formatting. Such assistance should be documented as formatting or translation assistance, and it should not be used to generate, complete, or replace the mathematical reasoning.

(b) Economic reasoning questions

First work through the question entirely using your own reasoning and write your answer in the `.tex` file.

Once you have written your answer, you may ask AI to evaluate the reasoning and identify possible errors, omissions, or inconsistencies. You may ask AI to explain its criticism, but you must decide whether you agree with it and make any changes yourself.

Do not ask AI to generate, rewrite, or directly edit the economic reasoning in your answer. AI should serve as a **critic of an argument you have already developed**, rather than as the source of the argument.

After you have written the substantive answer yourself, you may ask AI to improve exposition, grammar, or LaTeX formatting. Such assistance should be documented as formatting or translation assistance, and it should not be used to generate, replace, or materially change the economic reasoning.

(c) Data analysis questions

For data analysis, you may use AI extensively to implement, optimize, integrate, and debug code. However, **the empirical design must come from you rather than from AI**.

Before asking AI to write code, you must create a file that specifies what the code should do. You may either:

- (i) describe the architecture of the analysis in plain English in a `.txt`, `.md`, or equivalent file; or
- (ii) write the initial code yourself in whatever programming language you choose.

Your specification must be sufficiently detailed that AI is **implementing your empirical design rather than making economically or econometrically substantive decisions for you**. In particular, whenever relevant, you should specify the data sources, sample restrictions, variable definitions, timing conventions, transformations, econometric specifications, standard error procedures, and desired outputs.

For example, a specification might contain the following (this is an overly simplistic example):

- i. Read the CRSP and COMPUSTAT data using my WRDS account.
- ii. For each firm i and each year t in COMPUSTAT, subset the data to firms satisfying the following criteria: [describe the criteria].
- iii. For each i and t in the resulting panel, create a BM variable, defined as the book equity to market equity ratio, that combines accounting information from the fiscal year ending in calendar year $t - 1$ with market capitalization information from CRSP as of December of year $t - 1$. BM is a ratio. The numerator is book equity (BE), calculated as follows: [explain the calculation or provide the precise reference containing the definition that AI should implement]. The denominator is market equity (ME), calculated as follows: [explain the calculation or provide the precise reference containing the definition that AI should implement].
- iv. For each i and t , align the BM variable with CRSP monthly returns for stock i from July of year t through June of year $t + 1$.
- v. Estimate Fama and MacBeth regressions of returns on BM using the resulting panel. Apply Newey and West (1987, 1994) when constructing standard errors.
- vi. Output the results as a table in a `.tex` file called **Question 1(a)**.

Once this specification has been created, you may ask AI to translate it into optimized and fully functional code, integrate that code with the rest of your project, run it, and debug implementation errors.

AI may make **programming choices** that do not alter the substance of your empirical design. For example, choosing efficient data structures, functions, packages, loops versus vectorized operations, or ways to organize the code.

AI may **not silently make substantive choices that are missing from your specification**. If implementation requires an economically or econometrically meaningful decision that you did not specify (e.g., a sample restriction, timing convention, treatment of missing observations, variable definition, winsorization rule, regression specification, or standard error choice), you should instruct AI to identify the ambiguity rather than resolve it for you. You must decide how to address the issue and update your specification before asking AI to implement the decision.

You may also use AI to debug code. If the problem is purely computational, AI may fix it directly. If debugging reveals that an empirical design decision is required, you must make that decision yourself and update the specification before AI implements it.

The principle is simple: **you design the empirical analysis; AI can only implement it.**

2. How to document your AI usage

If you use AI, your Git repository must also contain a complete, auditable record of that use.

(a) Use a single AI project/workspace

All AI use related to the problem set must occur within **one project/workspace associated with that problem set** in an environment with AI such as VS Code, Codex, or an equivalent application.

Allowed environments include Codex, VS Code with an AI agent, Cursor, or an equivalent coding or writing environment with AI, provided that the environment is associated with the problem set and its activity can be connected to the submitted repository and documentation. You may use different models or agents within the designated project only if their use remains part of the same auditable project record.

You may not use AI for the problem set through separate chatbot conversations, temporary chats, other AI applications, or other projects/workspaces outside the designated project. Examples of disallowed use include a separate temporary ChatGPT conversation, a separate Claude or Gemini chat, or another AI workspace whose interaction record is not part of the problem set's auditable project record.

AI inline autocomplete, grammar or style tools, and ordinary IDE code completion are allowed.

Routine autocomplete, spelling, grammar, or formatting suggestions do not require a separate entry in `AI_INTERACTIONS.md`. However, if an AI tool contributes substantive code, mathematics, economic reasoning, empirical choices, or answer text, its use must be captured through the Git history and summarized in `AI_USAGE.md`. If you deliberately ask an AI tool to produce, revise, critique, debug, or explain substantive work, treat that request as a substantive AI interaction under Section 2(c).

The purpose of this requirement is to ensure that all AI assistance associated with the problem set is observable within a single project.

(b) Keep the problem set files inside the AI project

The AI project/workspace should contain the Git repository and the files used to produce your problem set solution. This allows the AI agent to inspect your actual work and allows its actions to be linked to specific versions of your files.

(c) Use @TP for every substantive AI interaction

Before making any substantive request to AI, you must create the @TP skill using the prompt in Appendix A. You must invoke @TP at the beginning of every substantive AI interaction related to the problem set. Do not bypass @TP for substantive work. If the skill fails to create the required Git commits or `AI_INTERACTIONS.md` entry, stop and fix the record before continuing.

A request is substantive if it asks AI to check, critique, generate, revise, explain, debug, format, translate, summarize, or otherwise assist with content related to a problem set answer, derivation, empirical design, code, interpretation, or submission.

Purely administrative requests do not require separate Git snapshots before and after the request. Examples include asking AI to navigate files, explain how to run a terminal command, clarify something it already said without doing new work, or help with environment setup that does not affect the substance of the problem set.

Closely related minor subsequent requests may be grouped into one substantive AI interaction if they concern the same problem set item, occur in the same work session, and are documented together through @TP. For example, a short sequence of minor debugging iterations or formatting adjustments may be recorded as one interaction rather than as separate interactions for every prompt.

The @TP skill is responsible for creating the Git snapshots before and after the interaction and for appending the required entry to `AI_INTERACTIONS.md`. This procedure is intended to create a contemporaneous record of your AI use rather than relying on you (or the AI agent) to reconstruct that history after the problem set is finished.

(d) Do not alter the AI interaction record

Do not delete, rewrite, combine, or selectively omit entries in `AI_INTERACTIONS.md`. If an entry contains an error, leave the original entry in place and add a correction in a subsequent entry.

The Git history should therefore preserve both the evolution of your substantive work and the evolution of the AI interaction record.

(e) Create the final AI usage summary

After completing the problem set, give the AI agent in your project the following prompt:

Create a file called `AI_USAGE.md` that provides a complete, factual summary of how AI was used throughout this problem set.

Use `AI_INTERACTIONS.md`, the Git commit history, and the differences between the relevant Git commits as your primary evidence. Do not rely on memory of previous conversations when the repository provides a record, and do not infer the student's reasoning, intentions, or understanding when these are not directly supported by the available evidence.

Organize `AI_USAGE.md` first by problem set item and, within each item, chronologically by AI interaction.

For each AI interaction, report:

1. **Purpose:** What the student asked AI to do.
2. **Work before AI:** What relevant derivation, economic reasoning, empirical design, or code already existed in the repository immediately before the AI interaction. Base this description on the Git snapshot before the interaction.
3. **AI assistance:** What substantive assistance AI provided, including errors identified, explanations provided, code generated or debugged, and suggestions made.

4. **AI modifications:** Which files, if any, AI directly changed during the interaction and the nature of those changes.
5. **Substantive decisions:** Identify any mathematical, economic, or econometric choices suggested or made by AI. Distinguish these from purely mechanical programming or formatting choices.
6. **State after AI:** Describe the relevant differences between the Git snapshots before and after the interaction.
7. **Subsequent student work:** When observable from the Git history, describe changes the student made after the AI interaction and before the next AI interaction. Do not attribute a change to the student unless the Git history supports that attribution.
8. **Type of AI use:** Classify the interaction using all applicable categories: math review; economic reasoning review; empirical coding; code debugging; formatting or translation; or other.

If the available evidence does not support any of these items, write **“not shown in the available record”** rather than guessing.

At the beginning of `AI_USAGE.md`, also provide a short summary table listing, for each problem set item, the number of AI interactions and the main types of AI assistance used.

The purpose of `AI_USAGE.md` is to allow another AI agent or the instructor to evaluate how the student used AI. Accordingly, be descriptive and factual. **Do not evaluate whether the student’s AI use complied with the course policy; simply document what occurred.**

Do not delete or modify `AI_INTERACTIONS.md` or any other existing project files when producing `AI_USAGE.md`.

After the prompt above is complete, make sure to include both `AI_INTERACTIONS.md` and `AI_USAGE.md` in the Git repository you submit.

The grading of the problem set can be directly affected by the level of compliance with this AI policy, including the completeness, accuracy, and auditability of the AI use record.

Appendix A: Prompt for Creating the @TP Skill

Use the following prompt inside the designated AI project/workspace to create a reusable traceable prompt skill. After creating the skill, invoke @TP at the beginning of each substantive AI request related to the problem set.

Create a reusable AI skill called @TP, short for traceable prompt, for this problem set project. The purpose of this skill is to help me comply with the BUSFIN 8200 problem set AI policy. When I invoke @TP at the beginning of a substantive AI request, do the following.

1. First, identify the problem set item to which my request relates. If the item is not clear, ask me to specify it before doing the substantive work.
2. Before doing the substantive work, create a Git commit that records the current state of the repository. If there are no file changes to commit, create an empty commit so that the state before the interaction is still recorded. Record the commit hash for the state before the interaction.
3. Complete my substantive request, subject to the course AI policy. If the request asks you to make a substantive mathematical, economic, or empirical design decision that I have not specified, identify the ambiguity and ask me to decide before implementing it.
4. After completing the substantive work, append a new entry to `AI_INTERACTIONS.md`. Do not modify, delete, combine, or rewrite previous entries.
5. The new entry in `AI_INTERACTIONS.md` must include the problem set item; my substantive prompt; the purpose of the request; the Git commit before the interaction; a concise but complete description of the assistance you provided; the files you inspected; the files you directly modified, if any; any errors, omissions, or ambiguities you identified; any substantive mathematical, economic, or empirical suggestions you made; whether the interaction involved checking mathematics, checking economic reasoning, empirical implementation, code debugging, formatting/translation, or another form of assistance; and whether any minor

subsequent debugging or formatting requests were grouped into this same interaction.

6. After updating `AI_INTERACTIONS.md`, create another Git commit recording the state of the repository after the interaction. If there are no file changes to commit, create an empty commit so that the state after the interaction is still recorded.
7. If I explicitly group closely related minor subsequent debugging or formatting requests into the same interaction, document them in the same `AI_INTERACTIONS.md` entry. Only group subsequent requests when they concern the same problem set item, occur in the same work session, and are documented together.

The skill should help create an auditable record. It should not weaken or replace any requirement in the course AI policy.